



DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Clasificación binaria de lesiones pre-tumorales a partir de perfiles de expresión génica

Trabajo Práctico

27 de septiembre de 2026

Aprendizaje Automático I

Grupo 11

Integrante	LU	Correo electrónico
Bravo, Iván Agustín	786/23	bravoivanagustin@gmail.com
Kiss, María Delfina	72/23	kissdelfina@gmail.com
Lützow Holm, Eric	323/12	elholm90@gmail.com
Mendoza, Juan Cruz	627/23	juaninomendoza@gmail.com
Rostan, Marcos Andrés	589/23	rostanmarcos@gmail.com



Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

Ciudad Universitaria - (Pabellón Cero + Infinito)

Intendente Güiraldes 2610 - C1428EGA

Ciudad Autónoma de Buenos Aires - Rep. Argentina

Tel/Conmutador: (+54 11) 5285-9721 / 5285-7400

<https://dc.uba.ar>

1. Introducción

En este trabajo práctico evaluamos distintos modelos de aprendizaje automático para determinar si una muestra de células tiene pronóstico de crecimiento anómalo o descontrolado, condición conocida como *hiperplasia*. El dataset que usamos consta de 500 instancias provenientes de pacientes con lesiones pretumorales, en las que se midió la expresión de 200 genes. Cada instancia está etiquetada como verdadero (buen pronóstico) si no hubo indicios de una nueva *hiperplasia*, o falso (mal pronóstico) en caso contrario.

Con el fin de comparar su eficacia en la clasificación, probamos diferentes algoritmos con distintas configuraciones de hiperparámetros: árboles de decisión, K vecinos más cercanos (KNN), support vector machines (SVM), análisis discriminante lineal (LDA) y Naive Bayes. La implementación se realizó en Python siguiendo los lineamientos de la cátedra.¹

2. Separación de datos

Lo primero que hicimos fue dividir el dataset en dos particiones: un conjunto de desarrollo, sobre el cual entrenamos y seleccionamos los modelos, y un conjunto de evaluación (*held-out*) para estimar al final del trabajo el desempeño de la configuración elegida sobre los datos de control. Siguiendo el estándar habitual, destinamos el 15 % de las instancias al conjunto de evaluación. A la hora de hacer la división, tuvimos en cuenta que cada partición debe ser representativa del dataset. Es decir, la proporción de etiquetas original tiene que mantenerse en cada subconjunto, y el subconjunto de evaluación no debería presentar propiedades ausentes en el de desarrollo, ni a la inversa. Del total, 359 instancias (71,8 %) corresponden a la etiqueta 0 (mal pronóstico) y 141 (28,2 %) a la etiqueta 1 (buen pronóstico). Para preservar las proporciones aplicamos muestreo estratificado: agrupamos las instancias por etiqueta y extrajimos una muestra aleatoria del 15 % dentro de cada grupo mediante los métodos `groupby` y `sample` de Pandas. El resultado dio $0,15 \times 359 \approx 54$ instancias de etiqueta 0 y $0,15 \times 141 \approx 21$ de etiqueta 1, lo que deja un conjunto de evaluación de 75 instancias y un conjunto de desarrollo de 425. Con la separación establecida, construimos los modelos.

3. Construcción de árboles de decisión

Buscamos construir y evaluar modelos de árboles de decisión, obteniendo una estimación realista de su performance. Como primer paso separamos el conjunto de desarrollo en entrenamiento y validación de manera proporcional a sus etiquetas, usando `train_test_split` de `sklearn` estratificado, y nos quedamos con un 0,8 de entrenamiento y 0,2 de validación. Sobre esa partición entrenamos un árbol con altura máxima 3 y el resto de los hiperparámetros en sus valores por defecto (criterio de corte Gini). De este árbol obtuvimos una accuracy de 0,6471 sobre el subconjunto de validación. Este valor sale de una única partición de 85 instancias, por lo que depende bastante de cuáles hayan caído del lado de validación. Para obtener una estimación más estable repetimos la medición con validación cruzada sobre el conjunto de desarrollo completo, con un K-fold estratificado de 5 particiones de manera que cada fold conserve la proporción de etiquetas. Para cada fold calculamos accuracy, area under the precision-recall curve (AUPRC) y area under the receiver operating characteristic curve (AUCROC), y además el score global utilizando únicamente los folds de validación. Los resultados se pueden observar en la Tabla 1.

Permutación	Acc (train)	Acc (val)	AUPRC (train)	AUPRC (val)	AUCROC (train)	AUCROC (val)
1	0,8265	0,6000	0,7153	0,3265	0,8626	0,5697
2	0,8265	0,7412	0,6862	0,4213	0,8037	0,6411
3	0,8353	0,6235	0,6981	0,2895	0,8506	0,4754
4	0,8265	0,6471	0,6900	0,3594	0,8429	0,6117
5	0,7971	0,6824	0,6491	0,3574	0,8538	0,5683
Promedios	0,8224	0,6588	0,6878	0,3508	0,8427	0,5732
Global	(NO)	0,6588	(NO)	0,3404	(NO)	0,5610

Tabla 1: Métricas de un árbol de altura máxima 3 con criterio Gini, por fold de la validación cruzada estratificada de 5 particiones sobre el conjunto de desarrollo. Global se calcula sobre las predicciones acumuladas de los 5 folds de validación.

¹El notebook con el código completo se puede consultar aquí. En este informe incluimos solo el código necesario para ilustrar la elección de hiperparámetros de los modelos.

Para interpretar estos valores necesitamos un punto de comparación, así que evaluamos con la misma validación cruzada un clasificador que ignora los atributos y predice siempre la clase mayoritaria. Este alcanza un accuracy de 0,7176, un AUPRC de 0,2824 que no es otra cosa que la proporción de positivos, y un AUCROC de 0,5000 porque asigna el mismo score a todas las instancias. De la Tabla 1 vemos que el modelo base, con altura máxima 3 y criterio Gini, logró un accuracy de validación de 0,6588, por debajo del 0,7176 del clasificador trivial. Su AUCROC de validación es 0,5732 en promedio y 0,5610 global, por encima del 0,5000 de referencia, y su AUPRC de 0,3508 mejora un poco el 0,2824 que supone predecir siempre la clase mayoritaria. El modelo base entonces casi no captura la clase minoritaria, aunque la brecha con entrenamiento es amplia: 0,8224 contra 0,6588 en accuracy y 0,8427 contra 0,5732 en AUCROC.

Luego exploramos 6 combinaciones de altura máxima y criterio de corte, manteniendo validación cruzada con $K = 5$ y generando las combinaciones con `ParameterGrid` de scikit-learn. Probamos alturas máximas de 3, 5 e infinita, con criterios de corte Gini y Entropía (Tabla 2). Reutilizamos las mismas particiones para que los valores sean comparables. Por ende, la fila de altura 3 con Gini reproduce exactamente el resultado de la Tabla 1.

Altura máxima	Criterio de corte	Accuracy (training)	Accuracy (validación)
3	Gini	0,8224	0,6588
5	Gini	0,9482	0,6447
Infinito	Gini	1,0000	0,6400
3	Entropía	0,8029	0,6541
5	Entropía	0,9253	0,6612
Infinito	Entropía	1,0000	0,5953

Tabla 2: Accuracy promedio en entrenamiento y validación de árboles de decisión, según altura máxima y criterio de corte, con la misma validación cruzada estratificada de 5 particiones sobre el conjunto de desarrollo.

En la Tabla 2 se observa el comportamiento típico del overfitting. Al relajar la altura máxima, la accuracy de entrenamiento sube hasta 1,0000, es decir que el árbol memoriza las 340 instancias de cada fold, mientras que el de validación no logra mejoras significativas. Con Gini el mejor resultado se da con altura 3 (0,6588) para luego caer a 0,6400 sin restricción, y con Entropía el mejor caso es el árbol de altura 5 (0,6612), empeorando en la configuración sin restricción.

De la exploración anterior se pueden extraer dos conclusiones principales. Primero, el criterio de corte tiene un impacto menor que la altura máxima: para una misma profundidad, las diferencias en accuracy de validación no superan los 2 puntos porcentuales. Segundo, aumentar la profundidad mejora el ajuste sobre entrenamiento pero no sobre validación, lo que es una señal clara de sobreajuste: el árbol sin restricción de altura memoriza el conjunto de entrenamiento pero su validación cae a 0,6400 con Gini y 0,5953 con Entropía.

Ninguna de las 6 configuraciones supera el 0,7176 del clasificador trivial, y el rango completo de la columna de validación, que va de 0,5953 a 0,6612, es más angosto que la variación entre folds de una misma configuración, que en la Tabla 1 va de 0,6000 a 0,7412. Por eso no consideramos que ninguna combinación sea preferible a las demás. Tampoco la accuracy es la métrica más adecuada a este problema, porque con casi 71,8 % de instancias negativas premia al modelo que ignora la clase minoritaria y por eso de acá en adelante usamos AUCROC. En conjunto, las dos tablas muestran que un árbol de decisión no alcanza para estos 200 atributos, lo que motiva explorar otras familias de algoritmos.

4. Comparación de algoritmos

En esta sección evaluamos y comparamos modelos basados en árboles de decisión, KNN y SVM, explorando distintas combinaciones de hiperparámetros en cada caso mediante `RandomizedSearchCV` con 50 iteraciones y seleccionando las mejores configuraciones según el AUCROC promedio en validación cruzada estratificada de 5 folds sobre el conjunto de desarrollo.

4.1. Árboles de decisión

Como hiperparámetros consideramos la profundidad máxima, la cantidad mínima de instancias necesarias para dividir un nodo, la cantidad mínima de instancias por hoja, el criterio de corte (Gini o

Entropía) y la cantidad máxima de atributos a evaluar (probando la raíz cuadrada y el logaritmo del total de variables):

```
parametros_arbol = {"max_depth": [3, 5, 10, 20, None],
                    "min_samples_split": [2, 5, 10, 20],
                    "min_samples_leaf": [1, 5, 10, 15],
                    "criterion": ["gini", "entropy"],
                    "max_features": ["sqrt", "log2", None]}
```

Los mejores hiperparámetros encontrados fueron: 'min_samples_split': 10, 'min_samples_leaf': 15, 'max_features': 'log2', 'max_depth': 5, 'criterion': 'gini', obteniendo un AUCROC score de 0,6740.

4.2. KNN

Para KNN es importante considerar el escalado de las variables. Al tener escalas muy distintas, aquellas con mayor rango numérico pueden llegar a dominar el cálculo de la distancia. Los hiperparámetros evaluados fueron:

- **n_neighbors**: mientras un k pequeño puede llegar a sobreajustarse al ruido local de cada instancia, un k grande tiende a aproximarse a predecir la clase mayoritaria.
- **weights**: permite asignar mayor peso a los puntos más cercanos.
- **metric**: define la métrica de distancia utilizada para medir la cercanía.
- **scaler**: define el uso de escalador estándar o no.

```
parametros_knn = {"knn__n_neighbors": [3, 5, 7, 10, 15, 21, 31, 41, 51, 61, 75],
                  "knn__weights": ["uniform", "distance"],
                  "knn__metric": ["euclidean", "manhattan"],
                  "scaler": [StandardScaler(), None]}
```

Mejores hiperparámetros encontrados para KNN: 'scaler': None, 'knn__weights': 'distance', 'knn__n_neighbors': 21, 'knn__metric': 'manhattan', alcanzando un AUCROC de 0,8474.

4.3. SVM

Para SVM, al igual que en KNN, la escala de las características suele ser un factor crítico. Por eso, la consideramos junto a los siguientes hiperparámetros:

- **kernel**: determina si la frontera de decisión es lineal en el espacio original de las variables, o si se aplica un mapeo (*kernel trick*) para construir un hiperplano en un espacio de mayor dimensión.
- **C**: controla la regularización. Un valor mayor obliga al modelo a tener un margen estricto (lo que puede llevar a *overfitting*), mientras que un valor menor tolera más puntos mal clasificados.
- **degree**: aplica únicamente a kernels polinómicos y define el grado máximo del polinomio.
- **gamma**: para los kernels rbf y poly, define el radio de influencia de cada vector de soporte.

Para gamma y C se suelen probar diferentes órdenes de magnitud. Para degree, probar grados mayores a 3 puede generar un espacio de variables enorme y sobreajustar fácilmente con pocas instancias.

```
parametros_svm = [{"svm__kernel": ["linear"], "svm__C": [0.01, 0.1, 1, 10, 100, 1000],
                  "scaler": [StandardScaler(), None]},
                  {"svm__kernel": ["rbf"], "svm__C": [0.01, 0.1, 1, 10, 100, 1000],
                  "svm__gamma": ["scale", 0.0001, 0.001, 0.01, 0.1, 1],
                  "scaler": [StandardScaler(), None]},
                  {"svm__kernel": ["poly"], "svm__C": [0.01, 0.1, 1, 10, 100, 1000],
                  "svm__degree": [2, 3], "svm__gamma": ["scale", 0.0001, 0.001, 0.01, 0.1, 1],
                  "scaler": [StandardScaler(), None]}]
```

Los mejores hiperparámetros encontrados para SVM fueron 'svm__kernel': 'poly', 'svm__gamma': 1, 'svm__degree': 2, 'svm__C': 1000, 'scaler': None, con un AUCROC de 0,8587.

Se observa en la Tabla 3 que las 5 mejores configuraciones obtienen exactamente el mismo AUCROC. Al no aplicar escalado, los datos presentan rangos muy grandes, por lo que el producto interno entre vectores domina el cálculo del kernel independientemente del valor de `C` y `gamma`, lo que explica que distintas combinaciones de estos hiperparámetros resulten en el mismo AUCROC.

Hiperparámetro / Métrica	Config. 1	Config. 2	Config. 3	Config. 4	Config. 5
Escalador (Scaler)	None	None	None	None	None
Kernel	Polinómico	Polinómico	Polinómico	Polinómico	Polinómico
Grado (degree)	2	2	2	2	2
Gamma (γ)	1	0,001	1	0,01	0,01
C	1000	1000	100	100	1
AUCROC (Validación)	0,8587	0,8587	0,8587	0,8587	0,8587
Desvío entre folds	0,0552	0,0552	0,0552	0,0552	0,0552

Tabla 3: Mejores configuraciones para SVM obtenidas mediante búsqueda de hiperparámetros.

Resulta muy llamativo ver que tanto para SVM y KNN, las mejores configuraciones de hiperparámetros según AUCROC fueron aquellas que no utilizaron el escalador. Esto indica que las features con un rango más grande terminaron siendo aquellas que también tenían mayor poder predictivo.

4.4. LDA y Naive Bayes

Por último, añadimos a la comparación LDA y Naive Bayes (Tabla 4).

Fold	LDA	Naive Bayes
1	0,7131	0,7425
2	0,6919	0,7582
3	0,6291	0,7589
4	0,6086	0,6926
5	0,5608	0,7357
Promedio	0,6407	0,7376

Tabla 4: AUCROC por fold y promedio para LDA y Naive Bayes.

Observamos que el AUCROC promedio de LDA resulta ser el más bajo de los modelos evaluados, con un valor de 0,6407. Esto puede deberse a que se está estimando una matriz de covarianza de 200×200 (debido a las 200 variables utilizadas en el entrenamiento) con pocas instancias por fold, lo que resulta en una estimación ruidosa y casi singular. Para mitigar este problema en el futuro, se podría utilizar el hiperparámetro de `shrinkage`, el cual regulariza la matriz de covarianza.

Por otro lado, Naive Bayes, incluso sin que hayamos realizado una búsqueda de hiperparámetros, alcanzó un AUCROC promedio de 0,7376, un desempeño muy competitivo que se acerca a los resultados obtenidos por SVM y KNN. Al asumir independencia entre las variables, este modelo solo estima la media y la varianza para cada gen individualmente, por lo que dichas estimaciones resultan más robustas incluso contando con pocos datos. Uno de los hiperparámetros que se podría ajustar para optimizarlo es `var_smoothing`, el cual añade a todas las estimaciones una fracción de la mayor varianza observada en el conjunto de datos. Esto funciona como una regularización y evita que si una variable presenta una varianza cercana a cero en el entrenamiento, un valor nuevo en la fase de prueba reciba una probabilidad nula y termine dominando el producto de probabilidades.

Una comparativa que resume las mejores configuraciones para los cinco algoritmos probados se puede ver en la Tabla 5.

Algoritmo	Mejor Configuración / Hiperparámetros	AUCROC
Árbol de Decisión	<i>altura = 5, min_split=10, leaf=15, max_feat=log2</i>	0,6740
KNN	<i>k = 21, Métrica: Manhattan, Pesos: Distance, Scaler: None</i>	0,8474
SVM	Kernel: <i>poly</i> ($d = 2$), $\gamma = 1$, $C = 1000$, Scaler: <i>None</i>	0,8587
LDA	Por defecto	0,6407
Naive Bayes	Por defecto (<i>GaussianNB</i>)	0,7376

Tabla 5: Comparación general del mejor desempeño (AUCROC promedio de validación) por cada familia de algoritmos evaluada.

5. Evaluación de performance

El mejor modelo que encontramos en base a nuestros datos de desarrollo, medido según el AUCROC promedio de validación cruzada (0,8587) frente a las otras cuatro familias de algoritmos evaluadas, es el SVM sin escalar, con kernel polinómico de grado dos, gamma de 1 y regularización C de 1000. El SVM funciona bien con alta dimensionalidad, ya que no depende directamente de la cantidad de dimensiones (200) sino de la cantidad de datos en o cerca de los márgenes. Asimismo, el uso de un kernel no lineal permite capturar interacciones complejas y no lineales entre los niveles de expresión de distintos genes, superando las limitaciones de los modelos lineales tradicionales.

En particular, el kernel polinómico de grado 2 permite modelar directamente interacciones del tipo $x_i \cdot x_j$ entre pares de genes, lo cual explica su ventaja frente al resto de las familias evaluadas: LDA y el SVM lineal por construcción solo capturan combinaciones lineales, mientras que Naive Bayes puede modelar términos cuadráticos individuales (x_i^2) pero, al asumir independencia entre atributos, no logra capturar interacciones cruzadas entre variables. Por su parte, los árboles de decisión realizan cortes sobre un único atributo por vez, por lo que necesitarían demasiadas particiones sucesivas para aproximar una frontera sobre $x_i \cdot x_j$. Finalmente, KNN, al no ser paramétrico, sí es capaz de detectar estas relaciones locales, aunque su desempeño se ve afectado por los atributos más ruidosos y sin poder predictivo al calcular distancias sobre las 200 variables.

Para aproximar el desempeño de este modelo sobre el conjunto de control (en posesión de los docentes), entrenamos el modelo final utilizando la totalidad de las instancias del conjunto de desarrollo y, posteriormente, evaluamos sobre el conjunto de evaluación que fue separado al inicio del trabajo. De esta forma, logramos una estimación del desempeño final del modelo ante datos no vistos previamente, evitando sobreestimar su capacidad predictiva.

El valor de AUCROC estimado por el modelo final sobre este conjunto de evaluación fue de **0,9189**. Las probabilidades predichas para la clase positiva, correspondientes a cada instancia del conjunto de control, se encuentran adjuntas a este trabajo en el archivo `11_y_pred_control_9189.csv`.

6. Discusión

Es importante notar que el problema que abordamos concierne a la salud humana. Los modelos que intentamos encontrar buscan predecir lo mejor posible si una célula tendrá pronóstico de generar un tumor maligno o no. En este dataset, los casos negativos son los de mayor importancia, ya que implican que la persona a la que se le tomó la muestra debe realizarse estudios que confirmen el mal pronóstico o eventualmente iniciar un tratamiento. Así, aún sabiendo que es imposible generar un modelo perfecto dada la gran variabilidad que hay dentro de las muestras biológicas, podríamos tener en cuenta que cometer un error al clasificar una célula con mal pronóstico real como de buen pronóstico podría tener consecuencias mucho más graves que si el error se comete a la inversa.

Por otro lado, exploramos qué sucede al restringir el modelo a los atributos con mayor varianza, en lugar de usar las 200 variables. Esta decisión estuvo motivada por el hecho de que las mejores configuraciones de KNN y SVM no utilizaron escalado, lo que sugiere que los atributos con mayor rango numérico son también los más informativos. Al entrenar el SVM polinómico de grado 2 únicamente sobre el subconjunto de 55 variables de mayor varianza, obtuvimos una mejora en el AUCROC promedio de validación cruzada (pasando de 0,8587 a 0,9441) respecto al mismo modelo entrenado con las 200 variables. Además, al comparar distintos tipos de kernels bajo esta misma métrica, notamos que el AUCROC de validación se disparaba a 0,9421 al incluir términos cruzados $x_i \cdot x_j$ respecto al kernel lineal (0,7190), mientras que los términos puramente cuadráticos x_i^2 por sí solos (0,6026) empeoraban el desempeño frente al caso

lineal. Esto refuerza la hipótesis de que las interacciones entre pares de genes son el principal mecanismo predictivo en este dataset, y sugiere que una selección de features más sistemática podría mejorar aún más el desempeño y la interpretabilidad del modelo.

Finalmente, cabe aclarar que el AUCROC obtenido sobre el conjunto de evaluación *held-out* fue de 0,9189, notablemente superior al 0,8587 de validación cruzada sobre los datos de desarrollo. Esta diferencia llama la atención porque va en la dirección opuesta a la esperada: lo habitual es que el desempeño sobre datos no vistos sea igual o inferior al estimado por validación cruzada, no superior. Una explicación posible es el tamaño reducido del *held-out* (75 instancias), que genera una estimación con alta varianza: con tan pocos datos, el desvío estándar del AUCROC puede ser del orden de 0,05 a 0,10, por lo que un valor de 0,9189 es perfectamente compatible con el 0,8587 de validación cruzada dentro del margen de error. Esto ya se intuía al observar un desvío de 0,0552 entre los folds sobre 425 instancias; con un conjunto cinco veces más chico, esa varianza es considerablemente mayor. En consecuencia, no interpretamos el 0,9189 como evidencia de que el modelo generaliza mejor de lo estimado, sino como una fluctuación estadística esperable dado el tamaño del conjunto de evaluación.

Para analizar cuánto podría beneficiarse el modelo de contar con más datos de entrenamiento, construimos una curva de aprendizaje. Mezclamos aleatoriamente el conjunto de desarrollo y entrenamos la misma configuración (SVM con kernel polinómico de grado 2, $\gamma = 1$ y $C = 1000$) sobre proporciones crecientes de él (0,1; 0,2; ...; 1,0), midiendo en cada caso el AUCROC sobre las instancias de entrenamiento y sobre el conjunto de evaluación, que se mantuvo fijo. Dada la acotada cantidad de datos con la que contamos, es esperable que el modelo siga sobreajustando. El AUCROC en entrenamiento se mantiene en 1,0 para todos los tamaños, es decir que incluso con las 425 instancias de desarrollo el modelo sigue separando perfectamente los datos con los que fue entrenado, algo coherente con un valor de $C = 1000$ que penaliza fuertemente los errores de clasificación. En cambio, el AUCROC sobre el conjunto de evaluación crece de 0,60 con el 10 % de los datos hasta 0,92 con la totalidad, y no muestra señales de estabilizarse. La brecha entre ambas curvas sigue siendo amplia al final, por lo que todo indica que el modelo se beneficiaría de contar con más instancias. Las oscilaciones de la curva de evaluación, como la caída con el 50 % de los datos, son consistentes con la alta varianza que discutimos para un conjunto de solo 75 instancias. La curva de aprendizaje puede consultarse en la sección **Apéndice: curvas de aprendizaje** del notebook adjunto.

7. Conclusiones

A lo largo de este trabajo evaluamos cinco algoritmos de clasificación sobre un dataset de expresión génica de 200 variables y 500 instancias, con el objetivo de predecir el pronóstico de lesiones pretumorales. La principal dificultad encontrada fue el desbalance de clases (71,8 % negativas), que hacía que la accuracy no fuera una métrica representativa del desempeño real de los modelos frente a un clasificador trivial que predice siempre la clase mayoritaria, por lo que centramos la selección y comparación de modelos en el AUCROC promedio de validación cruzada.

Los árboles de decisión, incluso con búsqueda de hiperparámetros, no lograron superar el clasificador trivial en accuracy y mostraron un AUCROC de validación de apenas 0,6740, lo que sugiere que la frontera de decisión de este problema no es fácilmente aproximable con cortes univariados sucesivos sobre 200 atributos. LDA tampoco se adaptó bien al problema, probablemente por la dificultad de estimar una matriz de covarianza de 200×200 con pocas instancias por fold. En el otro extremo, el SVM con kernel polinómico de grado 2 alcanzó el mejor AUCROC de validación de 0,8587, lo que sugiere que la frontera de decisión está gobernada principalmente por interacciones de pares de genes. Naive Bayes, con un AUCROC de 0,7376 sin ajuste de hiperparámetros, refuerza la idea de que la información predictiva está distribuida de manera relativamente independiente entre las variables.

Un aspecto que decidimos explorar fue la selección de atributos por varianza: al restringir el entrenamiento a las 55 variables de mayor rango, el AUCROC promedio en validación cruzada del SVM mejoró de 0,8587 a 0,9441 respecto al modelo entrenado con los 200 atributos originales, lo que es coherente con el hallazgo de que las mejores configuraciones sin filtrar no utilizaron escalado. Podría ser valioso explorar métodos de selección de features más sistemáticos y ajustar los hiperparámetros de Naive Bayes y LDA, que en este trabajo se evaluaron solo con sus valores por defecto.

8. Bibliografía

A continuación dejamos la documentación consultada de sklearn:

- *Tuning the hyper-parameters of an estimator.*
- RandomizedSearchCV.
- DecisionTreeClassifier.
- KNeighborsClassifier.
- SVC.
- LinearDiscriminantAnalysis.
- GaussianNB.